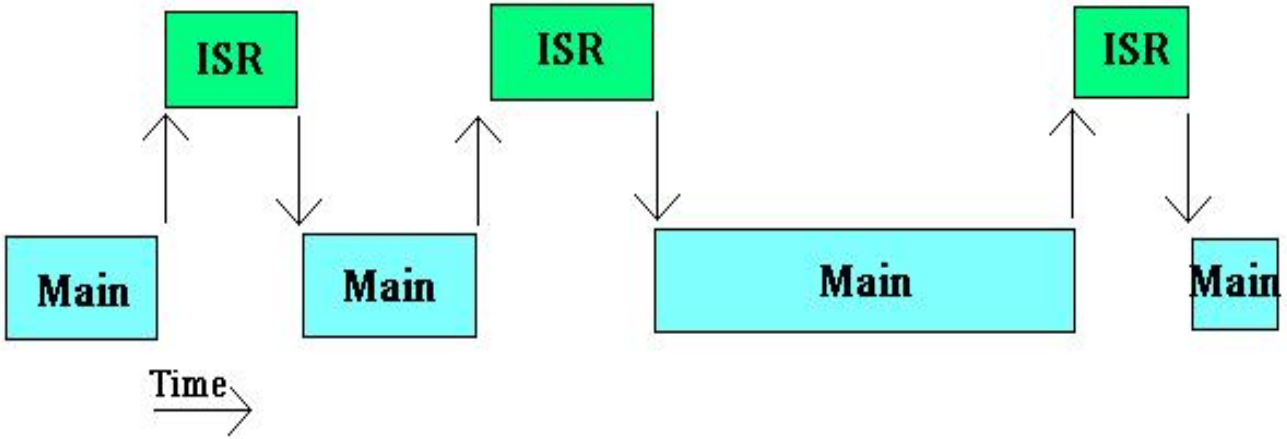
MICROPROCESSORS & MICROCONTROLLER
Interrupts

INTERRUPT

- An interrupt is an **event** which **breaks** the **normal execution** of the program.
- While the CPU is executing a program, an interrupt breaks the normal execution of instructions, **diverts** its **execution to some other program** called **Interrupt Service Routine(ISR)**.
- **After** executing the **ISR**, the **control** is again **transferred to the main program** which was being executed at the time of interruption.

Program execution without intrrupts :

Program execution with intrrupts :



ISR : Intrrupt Service Routin

INTERRUPT VS POLLING

1. Interrupts

- Whenever any device needs its service, the device notifies the microprocessor by sending it an **interrupt signal**.
- Upon receiving an interrupt signal, the **microprocessor interrupts** whatever it is doing and serves the device.
- The program which is associated with the interrupt is called the **Interrupt Service Routine (ISR)** or interrupt handler.

2. Polling

- The microprocessor **continuously monitors** the status of a given device.
- When the **conditions met**, it performs the service.
- After that, it moves on to **monitor the next device** until every one is serviced.

- The **polling method is not efficient**, since it wastes much of the microprocessor's time by polling devices that do not need service.
- The **advantage of interrupts** is that the microprocessor can serve many devices (not all at the same time).
- Each devices can get the attention of the microprocessor based on the **assigned priority**.
- For the polling method, it is **not possible** to assign priority since it checks all devices in a round-robin fashion.
- The microprocessor can also **ignore (mask)** a device request for service in Interrupt.

- Maskable Interrupt
- Non- maskable interrupt
- **<u>Types of interrupt:</u>**
  - Hardware interrupt
  - Software interrupt

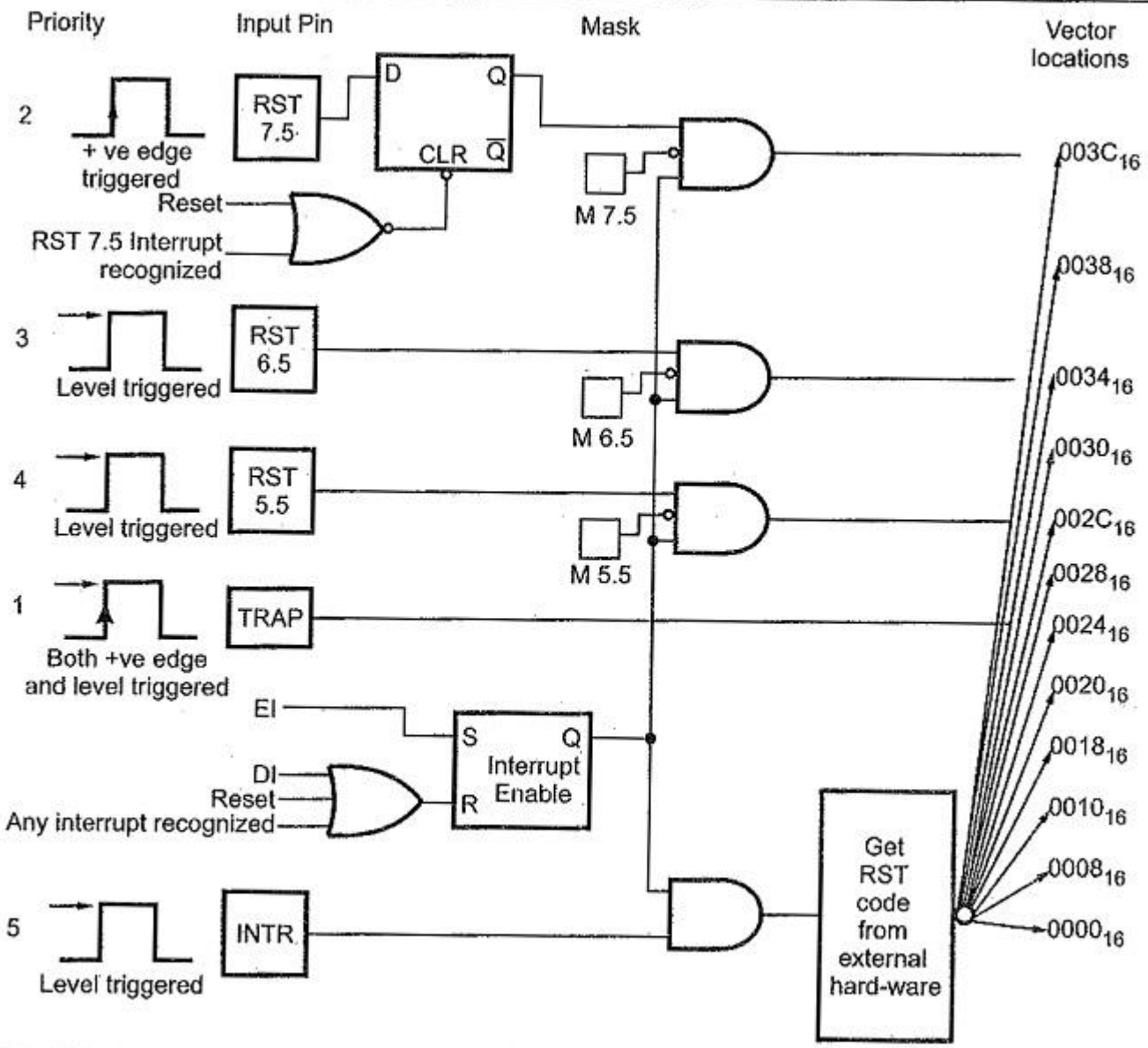
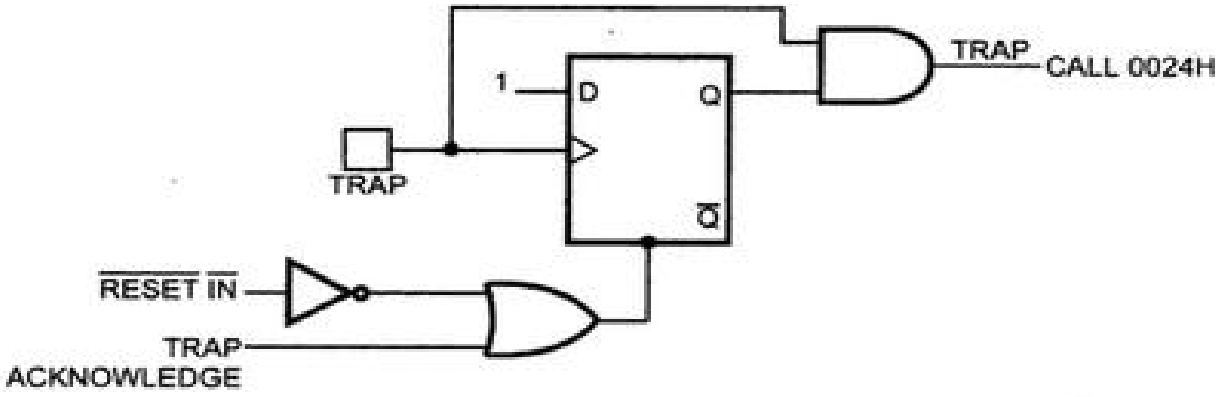


Fig. 5.2 Interrupt structure of 8085

HARDWARE INTERRUTPS OF 8085

- **<u>TRAP :</u>**
  - NMI
  - Unaffected by any mask or interrupt enable.
  - Edge and level triggered.
  - TRAP must go high and remain high until it is acknowledged.



- Two ways to clear TRAP
  - by resetting microprocessor (i.e) by giving low signal on RESETIN' pin.
  - By giving a high TRAP acknowledgement

- **<u>RST 7.5</u>**
  - Maskable interrupt.
  - 2nd highest interrupt
  - Positive edge triggered and it is stored internally by the D flip flop until it cleared by software reset using SIM instruction or by internally generating acknowledge signal.

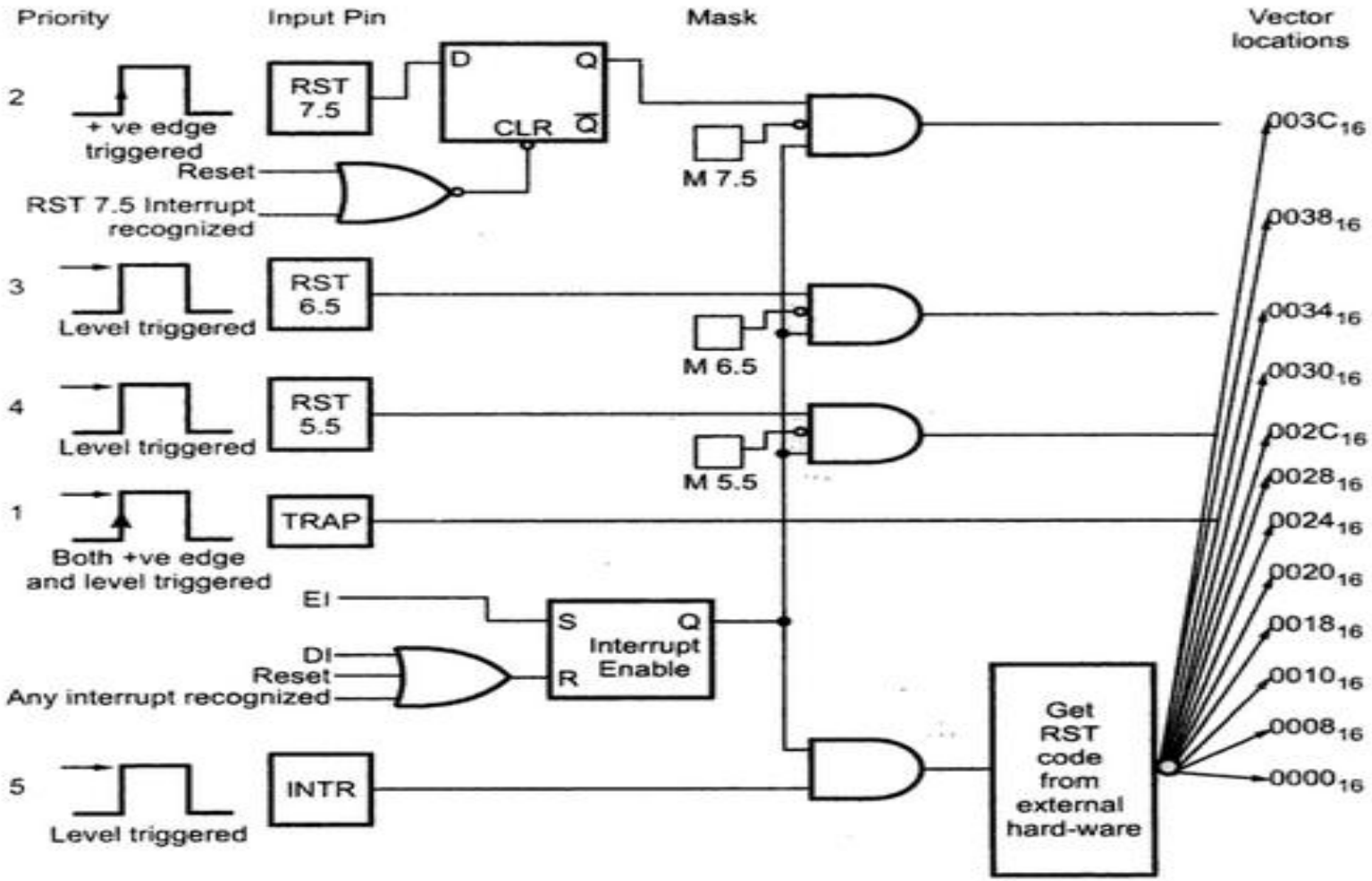
- **<u>RST 6.5 & RST 5.5</u>**
  - Both level triggered.
  - Masked using SIM instruction
  - 3rd and 4th priority respectively.
- **INTR**
  - Maskable but not vectored interrupt.
  - Following sequence will occur when INTR goes high
    - 8085 checks the status of INTR signal

- If INTR – high, then 8085 completes its current execution and sends an active low interrupt acknowledge signal, if the interrupt is enabled.
- In response to the INTA' signal, external logic places OPCODE on the data bus.
- On receiving the instruction, the 8085 saves the address of next instruction on stack and executes received instruction.

Interrupt type	Trigger	Priority	Maskable	Vector address
TRAP	Edge and Level	1st (Highest)	No	0024 H
RST 7.5	Edge	2nd	Yes	003CH
RST 6.5	Level	3rd	Yes	0034H
RST 5.5	Level	4th	Yes	002CH
INTR	Level	5th (Lowest)	Yes	

Table 5.1

INTERRUPT STRUCTURE



SOFTWARE INTERRUPT

- 8 interrupts
- RST 0 – RST 7

INTERRUPT NUMBER X 8 = VECTOR ADDRESS

Instruction	HEX code	Vector Address
RST 0	C7	0000H
RST 1	CF	0008H
RST 2	D7	0010H
RST 3	DF	0018H
RST 4	E7	0020H
RST 5	EF	0028H
RST 6	F7	0030H
RST 7	FF	0038H

MASKING/UNMASKIN INTERRUPTS

- EI
- DI
- SIM

PENDING INTERRUPTS

- RIM

https://www.eeeguide.com/types-of-interrupts-in-8085/